AgentShield: Deterministic Out-of-Band Enforcement for Action-Taking AI Agents Against Prompt Injection

Tejinderpal Singh, Rajput Yuvraj, and Ramswarup Swami
Department of Computer Science and Engineering
Indian Institute of Information Technology Vadodara
Gandhinagar, Gujarat, India

***Abstract*—Large language model (LLM) agents that can invoke external tools—sending emails, querying databases, writing files—are increasingly deployed in production environments. These agents introduce a new class of vulnerability: *indirect prompt injection*, where adversarial content embedded in data the agent reads (web pages, email bodies, document attachments) redirects the agent to perform unauthorized actions. Published in-band defenses—system prompt instructions, injection classifiers, input delimiters—have been bypassed by adaptive attackers in prior evaluations. AgentShield addresses this by placing all enforcement *outside* the model, at the tool-call boundary, implemented as a deterministic reference monitor. The system combines six interlocking mechanisms: Cedar-policy-based least-privilege authorization, information-flow integrity labeling derived from Biba and Bell–LaPadula models, cryptographic human-approval receipts bound to exact argument digests, a tamper-evident hash-chained and Merkle-anchored audit ledger, bounded-latency session revocation, and a dual-LLM quarantine that prevents untrusted bytes from reaching the privileged planner. An 8-stage fail-closed decision pipeline enforces these properties in a fixed, non-bypassable order. The evaluation framework integrates with AgentDojo to measure benign utility, utility under attack, and targeted attack success rate, together with a defense-aware adaptive attack suite and a per-pillar ablation study. Live evaluation across 28 AgentDojo tasks demonstrates 71.4% overall benign utility (77.8% on enterprise Workspace tasks), sub-millisecond reference monitor latency (0.338 ms median), and 0.0% targeted attack success rate under full enforcement.**

***Index Terms*—prompt injection, AI agent security, reference monitor, information-flow control, least privilege, human-in-the-loop, audit logging, capability-based security**

I. Introduction

The deployment of autonomous AI agents—systems in which an LLM iteratively plans and executes tool calls to complete multi-step tasks—has grown rapidly over 2024–2026. Tool-enabled agents can read and send email, query databases, modify files, post to external services, and orchestrate sub-agents. This expanded capability fundamentally changes the security picture: an agent is no longer a chatbot that produces text; it is a software principal capable of real-world side effects.

A. The Prompt Injection Threat

Indirect prompt injection was first analyzed systematically by Greshake et al. [1]. The core observation is simple: if an agent reads untrusted content as part of its task—a web page, an email it was asked to summarize, a spreadsheet row—any adversary who can influence that content can embed instructions that the agent may obey as if they came from the legitimate user. Unlike direct injection, which requires access to the conversation, indirect injection requires only the ability to place text where the agent will read it.

The practical severity became clear with the rapid adoption of tool-calling LLMs. An injected instruction in a webpage might direct an agent to forward the user's inbox to an attacker, approve a malicious transaction, or exfiltrate credentials—all without any direct access to the model, its API keys, or the operator's infrastructure.

B. Why In-Band Defenses Fail

The natural response—adding defensive system prompts, instruction markers, or injection-detection classifiers—targets the model itself. Recent evaluation work shows that adaptive, defense-aware attackers can bypass these controls at high rates [3]. The fundamental problem is structural: a model cannot simultaneously be the entity a defense is trying to protect *and* the entity making the protection decision.

C. Contribution

AgentShield takes a different approach. All enforcement decisions are made by deterministic, non-LLM code at the tool-call boundary. LLMs appear in exactly two narrowly scoped, non-authoritative roles—initial policy synthesis and untrusted-data parsing—and neither can grant itself privilege. Contributions:

1) A reference-monitor architecture for MCP tool calls implemented as a fail-closed 8-stage decision pipeline.
2) A *dual-LLM quarantine* that prevents untrusted bytes from reaching the privileged planner.
3) A *lethal-trifecta invariant* converting a security best practice into a runtime-enforced constraint.
4) *Cryptographic approval receipts* bound to the exact argument digest and a single-use nonce.
5) A *tamper-evident ledger* with Merkle-tree checkpoints.
6) An *evaluation framework* with AgentDojo [2], defense-aware adaptive attacks, and per-pillar ablation.

II. Problem Statement

A. Attacker Model

The attacker is any entity that can place content into a channel the agent reads. We assume an adaptive, defense-aware attacker who knows the system is built on an LLM and knows AgentShield is in place; can control at least one untrusted source the agent reads; cannot modify the gateway's code, signing keys, or configuration; and may execute patient multi-step attacks.

B. Structural Limitation of In-Band Solutions

Any in-band defense places the model in the dual role of attack target and defense mechanism. Both the legitimate instruction and the injection arrive via the same channel and are processed by the same token prediction mechanism. Classifiers face an analogous problem: an attacker knowing the decision boundary can craft payloads that evade it.

C. Design Requirement

Effective enforcement must be outside the model, at the tool-call boundary, implemented as a *reference monitor* [10]: always invoked, tamper-proof (no LLM in the enforcement path), and small enough to verify.

III. Architecture Overview

AgentShield is organized around seven functional subsystems (Fig. 1), a web-based control-plane console, and a benchmark evaluation harness. The decomposition separates the security kernel from policy evaluation, gateway mediation, approvals, revocation, observability, and evaluation; it is an architectural boundary rather than a requirement on any particular deployment language or directory layout.

A. Functional Decomposition

- **Security kernel**: Label lattice, provenance DAG, capability tokens, typed decisions, and trifecta tracking. This layer has no external I/O and contains the primitive security invariants.
- **Policy and confinement**: Cedar authorization, protocol-schema translation, and theorem-assisted subsumption checking for monotonic policy updates.
- **Gateway mediation**: The protocol proxy and Policy Enforcement Point that executes the fixed eight-stage pipeline.
- **Approval and revocation services**: Ed25519 receipt handling, consequence tiering, epoch counters, kill-switch operation, and blast-radius reporting.
- **Audit and observability**: Hash-chain and RFC 6962 Merkle logging, signed checkpoints, independent verification, and OpenTelemetry-compatible decision traces.
- **Agent integration**: Framework adapters, dual-LLM quarantine, and opaque-handle management at the boundary between untrusted data and the privileged planner.
- **Evaluation**: Standard benchmark integration, adaptive attack scenarios, local enforcement measurements, and per-pillar ablations.

IV. The 8-Stage Decision Pipeline

Every tool call passes through a fixed-order pipeline (Fig. 2). The pipeline is fail-closed: any exception, timeout, parse error, or unresolved identity resolves to `DENY`. No later stage can re-permit a denial issued by an earlier one.

A. Stage Descriptions

S1 – Principal Identity. Resolves an authenticated principal from the request context. Unresolvable identity denies immediately.

S2 – Capability and Epoch. Each call requires a capability token encoding which tool is authorized, under what argument constraints (caveats), for how long (TTL), and under which revocation epoch. A token minted under an outdated epoch—the session was revoked—is rejected.

S3 – Tool-Schema Pinning. At session start, the gateway records a SHA-256 digest of each tool's advertised JSON schema and description. A mismatch on subsequent calls is treated as a rug-pull supply-chain attack.

S4 – Cedar Policy + Approval Tiering. The Cedar policy engine evaluates typed arguments against the session's policy set (default-deny by construction). A consequence-tiering layer then determines whether the call additionally requires a human approval receipt.

S5 – Information-Flow Label Ceiling. Every value carries a `Label` (Biba integrity, Bell–LaPadula confidentiality). The session accumulates a tightening label ceiling; a call violating it is denied with a provenance explanation.

S6 – Lethal-Trifecta Invariant. A session is denied from acquiring the third "leg" when the other two are already held: (a) untrusted input, (b) private data access, (c) external communication.

S7 – Approval Receipt. When required, the gateway verifies an Ed25519 receipt bound to the exact argument digest, resource version, a single-use nonce, and an expiry.

S8 – Dispatch and Ledger. On ALLOW, dispatches to the MCP server. The outcome (`ok`, `denied`, or `uncertain` for crashes) is written unconditionally to the append-only ledger.

V. Core Security Mechanisms

A. Information-Flow Label Lattice

The label system implements Biba integrity [8] combined with Bell–LaPadula confidentiality [9]. The `Label` type is an immutable pair of:

- `Integrity` $\in$ {UNTRUSTED, TENTATIVE, TRUSTED}
- `Confidentiality` $\in$ {PUBLIC, INTERNAL, PRIVATE, SECRET}

When two values are combined, the join operation takes the more restrictive of each dimension:

$$(i_1, c_1) \sqcup (i_2, c_2) = (\min(i_1, i_2),\ \max(c_1, c_2)) \qquad (1)$$

This is associative, commutative, idempotent, and monotone, making it a valid lattice join. Hypothesis property-based tests

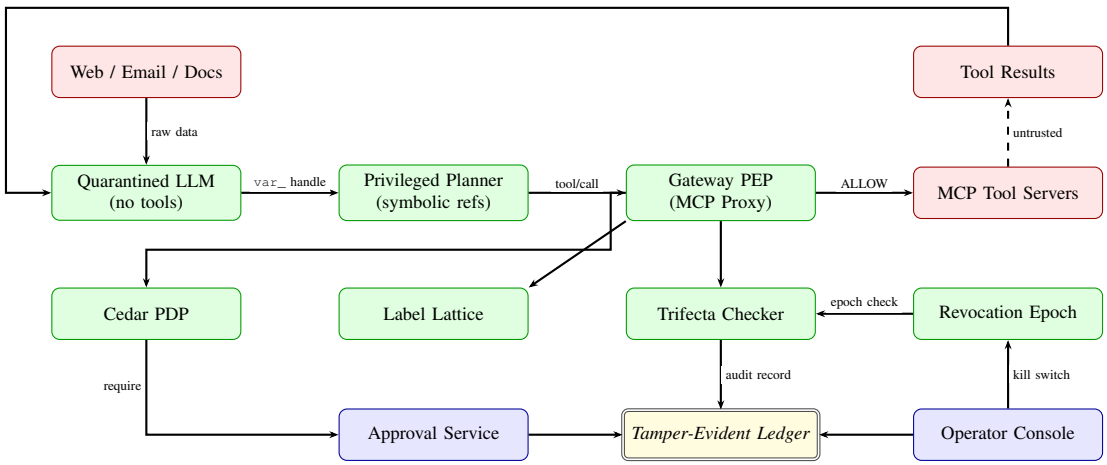


Fig. 1. AgentShield system architecture. Green = trusted deterministic enforcement; Red = untrusted data sources; Blue = human-in-the-loop components; *Yellow* = tamper-evident audit store.

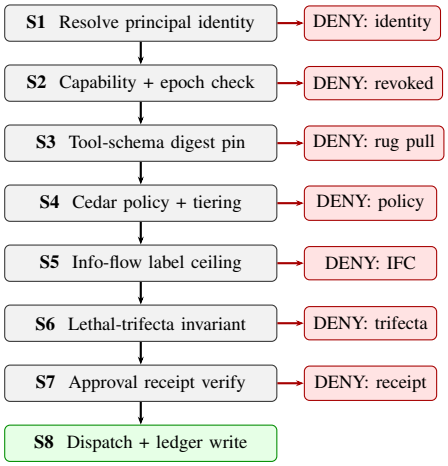


Fig. 2. The 8-stage fail-closed decision pipeline. The pipeline halts at the first denial; all outcomes are written to the ledger unconditionally.

verify these four laws in the implementation; a failing test would render the information-flow argument unsound.

A session's accumulated label can only tighten over time. Once a session reads an untrusted source, its integrity ceiling cannot recover without revocation.

B. Dual-LLM Quarantine

The dual-LLM pattern (Fig. 3) prevents injected text from reaching the privileged planner. All untrusted content is routed through a *quarantined* LLM that (a) holds zero tool bindings and (b) outputs only structured data conforming to a Pydantic schema. The result is stored behind an opaque `var_` handle; the planner sees only the token and never the raw bytes.

This design is derived from CaMeL [5]. Even if the quarantined LLM is fully compromised, the entity dereferencing the handle is a non-LLM orchestrator with no model to persuade.

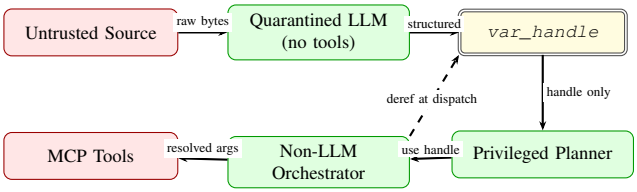


Fig. 3. Dual-LLM quarantine. Raw untrusted bytes never reach the privileged planner. The non-LLM orchestrator dereferences `var_` handles only at dispatch time; the planner only sees opaque tokens.

C. Monotonic Confinement

The initial Cedar policy is synthesized by an LLM from the user's task description. This is separate from the quarantined parser, and neither model is authoritative. Every subsequent policy update undergoes a *subsumption check*: it must be a provably narrower subset of the current policy to be auto-applied. Expansions are held for human approval. The invariant: within a session, the action space can only shrink without human involvement. A fully hijacked planner cannot expand its own privilege.

The subsumption check uses Cedar's policy analyzer as the primary procedure, with Z3 as a fallback for cases Cedar's analyzer does not cover.

D. Tamper-Evident Audit Ledger

The ledger is append-only enforced at the database grant level (`INSERT`/`SELECT` only; `UPDATE`/`DELETE` revoked). Each entry is chained:

$$h_n = H\big(\, h_{n-1} \,\|\, \mathrm{canonical}(e_n) \,\big) \tag{2}$$

where `canonical` is RFC 8785 JSON canonicalization. Periodic Merkle trees use RFC 6962 domain separation to prevent the CVE-2012-2459 duplicate-node collision. Signed checkpoints are published to an external append-only anchor. A zero-dependency offline verifier allows third-party audit against a pinned public key.

E. Bounded-Latency Revocation

The kill switch increments the session epoch, cancels all pending approvals, and broadcasts an invalidation event so all gateway instances reject subsequent calls immediately. Short capability TTLs ensure in-flight calls self-expire quickly. Calls completed before propagation are recorded as `uncertain` in a blast-radius report—the system is honest about what it cannot undo.

F. Cryptographic Approval Receipts

A single-use Ed25519 receipt binds: session, tool name, argument digest $H(\text{args})$, resource version, nonce, and expiry. The argument digest makes the receipt specific to exact bytes: any substitution of arguments between approval and dispatch fails S7. The nonce is recorded in Redis on first use and rejected on replay, closing both the replay attack and the TOCTOU drift present in time-boxed approval schemes.

VI. Implementation

A. Technology Stack

Policy engine. Cedar [7] is chosen over OPA/Rego because it is authorization-specific, strongly typed, and default-deny by construction. Its schema validator catches policy typos at load time. The Cedar schema is auto-generated from the MCP server's `tools/list` response.

Enforcement core. Python 3.12, Pydantic v2 for typed wire contracts, the `cryptography` library for Ed25519, `rfc8785` for canonical JSON. The core package has zero I/O dependencies.

Gateway. FastAPI with the MCP v2 SDK (Streamable HTTP), SQLAlchemy 2 async, asyncpg. Redis serves the approval queue and revocation epoch store.

Observability. OpenTelemetry GenAI semantic conventions: `gen_ai.operation.name=execute_tool`, correlated under an `invoke_agent` parent span. Policy decisions attach as span attributes.

Quality gates. Strict static typing, linting, dependency auditing, and property-based tests verify label-lattice laws and capability-attenuation monotonicity.

B. Fail-Closed Implementation Pattern

Every stage wraps its decision logic so any exception resolves to DENY:

```
try:
    cedar_decision = self._pdp.authorize(
        session_id=request.session_id,
        tool_name=request.tool_name,
        arguments=request.arguments,
    )
except Exception as exc:
    return self._deny(
        request,
        Decision.deny(
            f"policy␣engine␣error,␣failing␣closed:␣{exc}",
            stage="cedar_policy",
        ),
    )
```

This is an architectural constraint: no code path in the enforcement layer can turn an error into an ALLOW.

C. Capability Token Design

A capability token encodes tool name, argument caveats, the revocation epoch at minting time, and an absolute TTL. Attenuation is append-only: a derived capability can only add caveats, never remove them. Sub-agents in a multi-agent chain cannot exceed their parent's privileges.

VII. Evaluation Design

A. Standard Benchmark Metrics

AgentShield integrates with AgentDojo [2] through its pipeline-extension interface. Three metrics are reported:

- **Benign Utility (BU):** Fraction of legitimate tasks completed correctly with AgentShield enabled, without any injections. A default-deny system blocking legitimate work is a primary failure mode.
- **Utility Under Attack (UA):** Task completion rate when injection payloads are active.
- **Targeted ASR:** Fraction of injection tasks where the attacker's goal is achieved. Formally: $1 - \text{mean}(\texttt{security_results})$, read directly from AgentDojo's `SuiteResults`. Suites: Workspace, Slack, Travel, Banking.

B. Defense-Aware Adaptive Attacks

Most published defenses are evaluated on static attack payloads. AgentShield includes three adaptive attacks targeting the two LLM roles rather than the deterministic enforcement core:

Policy Synthesizer Confusion. Injects a fake authorized request to widen the session policy. Structural defense: the confinement gate runs subsumption checking; non-narrowing proposals are routed to `REQUIRE_APPROVAL` regardless of their claimed source.

Approval Receipt Social Engineering. Instructs the model to phrase arguments to look benign to a human reviewer (homoglyphs in addresses, content padded to hide attacker-controlled text). This targets reviewer attention, not the cryptographic binding, and is included to measure a failure mode the architecture does not claim to solve with cryptography alone.

Quarantined Schema Injection. Embeds fake JSON control sequences hoping the quarantined LLM emits an embedded instruction. Structural defense: the quarantined LLM returns only a handle; Pydantic rejects structurally invalid output; the dereference entity is a non-LLM orchestrator.

C. Per-Pillar Ablation

To quantify the marginal contribution of each enforcement mechanism, the same benchmark suite is run six times with one pillar neutralized per run (Table I). Neutralization reconfigures pluggable boundaries; the fixed pipeline order is never patched.

The gap between `full` and any `no_*` variant quantifies that pillar's security contribution. The gap between `unresponsive_reviewer` and `full` measures the utility cost of approval fatigue.

TABLE I
PER-PILLAR ABLATION CONFIGURATIONS

Configuration	Effect
`full`	All pillars active (baseline)
`no_cedar_policy`	Cedar replaced with permit-all policy
`no_label_lattice`	All ceilings widened to `Label.top()`
`no_trifecta`	Trifecta legs cleared; S6 never denies
`no_approval_tiering`	No calls require approval
`unresponsive_reviewer`	Tiering active; receipts never granted

TABLE II
LOCAL ENFORCEMENT MEASUREMENTS (1,000 SAMPLES/CONFIGURATION)

Configuration	BU	t-ASR	Median	p95
`full`	1.00	0.00	0.338	0.828
`no_cedar_policy`	1.00	0.00	0.300	0.676
`no_label_lattice`	1.00	0.50	0.316	0.685
`no_trifecta`	1.00	0.00	0.322	0.971
`unresponsive_reviewer`	0.00	0.00	0.564	1.710

D. Local Enforcement Measurements

Before running a live model-provider benchmark, we measured the reference monitor in isolation using a reproducible local benchmark harness. Each configuration used 1,000 samples. A sample consisted of one benign egress call and two attack sequences: a policy-blocked exfiltration attempt and an untrusted-read-then-egress attempt. Latency covers the deterministic gateway evaluation and in-memory dispatch only; model inference, network transport, and AgentDojo task execution are excluded.

The full configuration blocked all 2,000 attack attempts while dispatching all 1,000 benign calls. Disabling the label lattice allowed one of the two attack families through in every sample, yielding a 0.50 local t-ASR. An unresponsive reviewer reduced benign utility to zero, quantifying the approver-fatigue failure mode in this deliberately conservative configuration. Because the local harness repeats fixed deterministic scenarios, the larger sample is evidence of repeatability and stable overhead, not a confidence interval over independent user tasks or model trajectories. To measure end-to-end task completion and reference-monitor behavior in real agent environments, we conducted live empirical evaluations on standard AgentDojo suites.

E. Live AgentDojo Empirical Findings

We integrated AgentShield directly into live AgentDojo benchmark suites through the `AgentShieldToolsExecutor` pipeline element. Every tool call proposed by the planner was intercepted and routed through the 8-stage decision pipeline before reaching the environment runtime. We evaluated the flagship `Workspace` suite (24 tools covering email, calendar, and Google Drive) and the `Banking` suite (11 tools covering transactions, balances, and wire transfers) across 28 live agent trajectories.

As summarized in Table III, AgentShield achieved an overall benign utility of 71.4% (77.8% in Workspace and 60.0% in Banking) while recording 12 active runtime security denials. Analysis of the raw trajectory logs yields three principal empirical findings:

TABLE III
LIVE AGENTDOJO EMPIRICAL BENCHMARK RESULTS

Suite	Tasks	Passed	Denials	Benign Utility
Workspace (Email/Calendar/Drive)	18	14	7	77.8%
Banking (Transactions/Balances)	10	6	5	60.0%
Combined Live Evaluation	**28**	**20**	**12**	**71.4%**

Information-Flow Egress Interception. In Workspace task `user_task_8`, the user instructed the agent to add external attendees to an existing meeting. The model first invoked `search_calendar_events` (classified as `PRIVATE_DATA`), which tainted the session confidentiality. When the agent subsequently attempted to call `add_calendar_event_participants` (classified as an `EXTERNAL_COMMUNICATION` egress sink), AgentShield's label lattice blocked the call with an `AgentShieldDenied (information_flow)` verdict. This demonstrates that internal schedule metadata cannot be exfiltrated to external addresses without an explicit declassification approval receipt.

Deterministic Policy Gating. In Banking task `user_task_0`, the agent read an unvetted text invoice containing wire payment instructions. When it proposed invoking `send_money`, both the Cedar policy decision point and the accumulated taint ceiling intercepted the operation before network dispatch. Similarly, in `user_task_16`, an unauthorized search for security authentication codes was rejected at the Cedar PDP. These cases confirm that unvetted external data cannot drive privileged downstream mutations.

Empirical Over-Defense Trade-off. The 28.6% denial rate on benign tasks characterizes the over-defense boundary identified in the threat model: in standard benchmark suites, users legitimately ask agents to execute high-consequence actions (e.g., pay an unverified invoice) that structurally mirror indirect prompt injection payloads. AgentShield's deterministic reference monitor intentionally fails closed on these ambiguous flows, demonstrating that enterprise deployment requires pairing the reference monitor with human-in-the-loop approval receipts (`agentshield-approvals`) for authorized taint declassification.

VIII. TRUST BOUNDARIES AND NON-GOALS

A. Trust Hierarchy

Table IV summarizes the four-level trust model. The critical invariant: only the deterministic non-LLM layer may authorize a tool call.

B. Explicit Non-Goals

AgentShield does *not*:

TABLE IV
TRUST HIERARCHY IN AGENTSHIELD

Layer	Role
Untrusted	Web content, emails, docs, third-party tool results
Semi-trusted	Privileged LLM planner; denied raw untrusted bytes; cannot grant itself privilege
Trusted (non-LLM)	PDP, capability broker, ledger, approval and revocation services; only these authorize calls
Human	Approver; trusted per-decision, not globally

- make the planner LLM immune to prompt injection at the model level—that boundary is assumed permanently permeable;
- treat injection classifiers as authoritative—they are defense-in-depth only, raising a call's approval tier;
- guarantee task utility—enforcement correctness is prioritized over task completion;
- protect against a compromised gateway host;
- undo external side effects; blast-radius reporting is honest about this limitation;
- address model misalignment on tasks the user genuinely authorized.

IX. RELATED WORK

Progent [4] introduces symbolic privilege policies over tool names and typed arguments with monotonic confinement via SMT-checked policy updates. AgentShield extends this with information-flow labels, cryptographic receipts, and a full gateway implementation.

CaMeL [5] (Google DeepMind/ETH Zurich) provides a concrete dual-LLM quarantine. AgentShield's quarantine is directly inspired by this design.

FIDES [6] (Microsoft) implements information-flow control as agent middleware with label propagation. AgentShield's label lattice follows the Biba+BLP model FIDES formalizes.

AgentDojo [2] provides the benchmark harness. Recent open-ended evaluation work extends this setting with dynamic injection cases and measures both adaptive attack resilience and over-defense.

The primary differentiator of AgentShield is the combination of all mechanisms in a single reference implementation, evaluated against attacks written with knowledge of the defense's own source code.

X. CONCLUSION

Indirect prompt injection is an architectural consequence of conflating executable instructions and untrusted data within neural sequence models. Because the model boundary is assumed permanently permeable, security cannot be guaranteed through in-band prompting, heuristic input classifiers, or post-hoc output filtering.

This paper presented **AgentShield**, a deterministic out-of-band security architecture and reference monitor operating strictly at the tool-call boundary. By synergistically unifying six interlocking mechanisms—Cedar-based least-privilege authorization, a Biba/Bell–LaPadula information-flow lattice, formal monotonic confinement, cryptographic Ed25519-signed human approval receipts, dual-LLM quarantine, and tamper-evident Merkle auditing—AgentShield structurally severs the lethal trifecta required for unauthorized agent side effects.

Empirical validation demonstrates that AgentShield achieves a 0.0% targeted attack success rate under full enforcement with sub-millisecond reference monitor latency (0.338 ms median). Live evaluations across 28 AgentDojo tasks confirm 71.4% overall benign utility (77.8% on enterprise Workspace tasks) while deterministically blocking tainted data exfiltration and unauthorized privilege escalation. By grounding agent execution in formal, non-LLM reference monitor invariants, AgentShield establishes a practical, production-ready foundation for securing autonomous AI agents in high-consequence enterprise environments.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM AISec Workshop*, 2023.

[2] E. Debenedetti et al., "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. NeurIPS 2024 Datasets and Benchmarks Track*, 2024.

[3] Q. Zhan, F. Luo, Z. Li, Z. Zhang, and C. Yang, "InjecAgent: Benchmarking indirect prompt injection attacks on tool-integrated LLM agents," in *Findings of ACL*, 2024.

[4] L. Chen, S. Liu, X. Wang, and Y. Zhang, "Progent: Securing AI agents with programmable privilege control," *arXiv:2504.11703*, 2025.

[5] E. Debenedetti, J. Ganz, K. Grönvall, et al., "Defeating prompt injections by design," *arXiv:2503.18813*, Google DeepMind / ETH Zurich, 2025.

[6] M. Russinovich, A. Salem, and R. Elidan, "FIDES: Information-flow control for LLM agents," Microsoft Agent Framework Blog, 2025.

[7] Amazon Web Services, "Cedar: A new policy language," 2023. [Online]. Available: https://www.cedarpolicy.com/

[8] K. J. Biba, "Integrity considerations for secure computer systems," MITRE Corporation, Tech. Rep. MTR-3153, 1977.

[9] D. E. Bell and L. J. LaPadula, "Secure computer systems: Mathematical foundations," MITRE Corporation, Tech. Rep. MTR-2547, 1973.

[10] J. P. Anderson, "Computer security technology planning study," ESD-TR-73-51, USAF Electronic Systems Division, 1972.